

需求工程基础

软件工程与计算 II · 模块二 · 单元 1

需求是软件与现实世界的契约。

课程	软件工程与计算 II
专业	软件工程
课时	2 学时 · 需求模块第 1 讲

今日学习路径

① 为什么需求

需求是成败关键
经典失败案例

② 需求工程框架

需求工程的活动
开发与管理

③ 需求基础概念

需求到底是什么？
定义·本质·表述

④ 需求层次分类

业务/用户/系统级
功能与非功能

⑤ AI视角讨论

AI时代新挑战
能力边界讨论

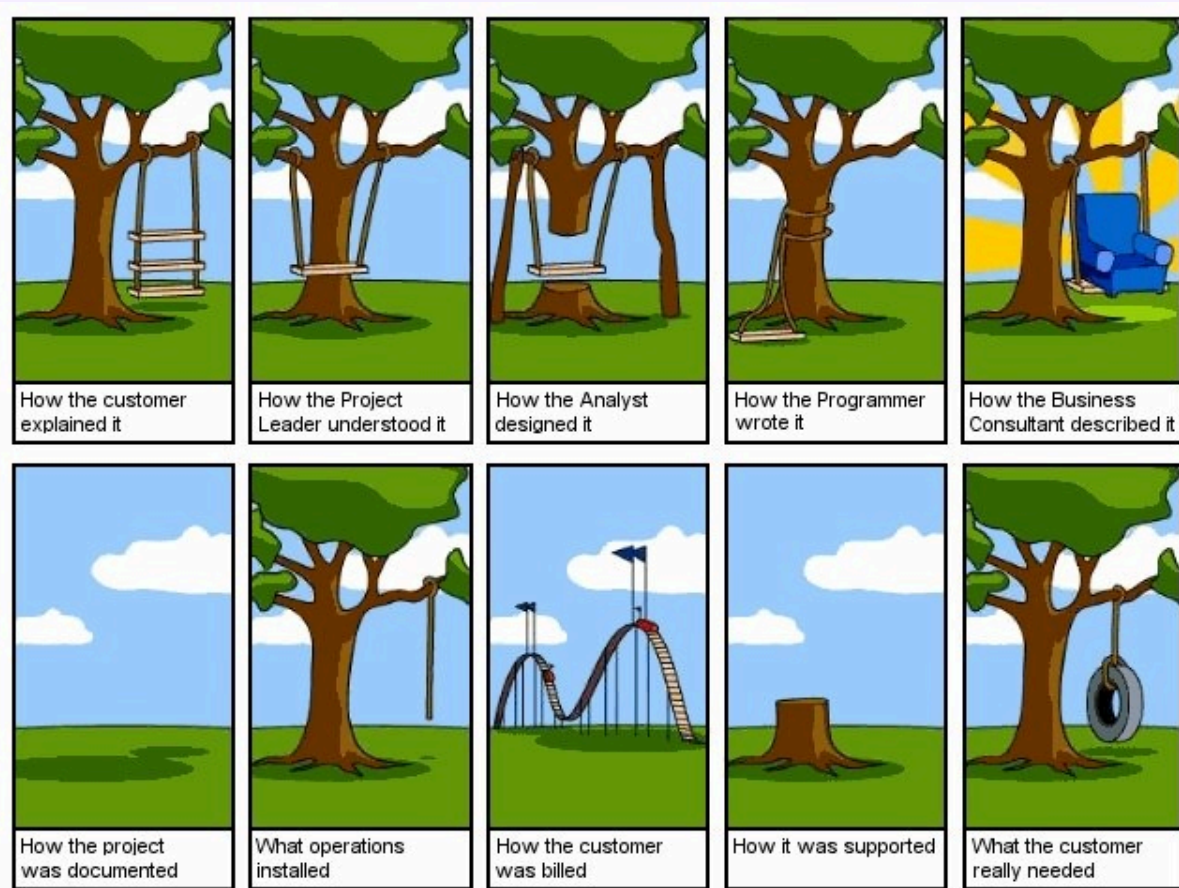


三个核心问题，今天结束时你应该能回答：

1. 为什么需求是软件**成败**的关键？
2. 需求到底是什么？有哪些**层次**？
3. AI 时代，需求工程面临什么**新挑战**？

PART 1 · 为什么要开发需求？

"What the customer wanted" —— 经典漫画



💡 这幅经典漫画揭示了一个核心事实：从客户到最终产品，每一个环节都可能产生理解偏差。需求工程的目标，就是尽可能缩小这些偏差。

产品需求为什么困难？

角色	关注点	表达方式
用户	业务目标、工作流程、操作习惯	自然语言、领域术语
开发	技术可行性、系统架构、实现成本	技术术语、模型图

- 用户和开发人员**背景不同**：知识领域、思维方式存在天然鸿沟
- 用户和开发人员**立场不同**：用户关注"解决什么问题"，开发关注"怎么实现"
- **信息不对称**：用户觉得"显而易见"的知识，开发可能完全不知道

💬 需求工程本质上是一座沟通的桥梁。

软件建立的依据

- 单纯的软件系统**不能解决问题**，它只有和现实世界形成**有效互动**才能实现问题的解决



🔍 软件开发的起点不是代码，而是对现实世界问题的理解。

需求工程的定义 [SWEBOK 1.1]

需求工程 (Requirements Engineering) 是所有需求处理活动的总和。

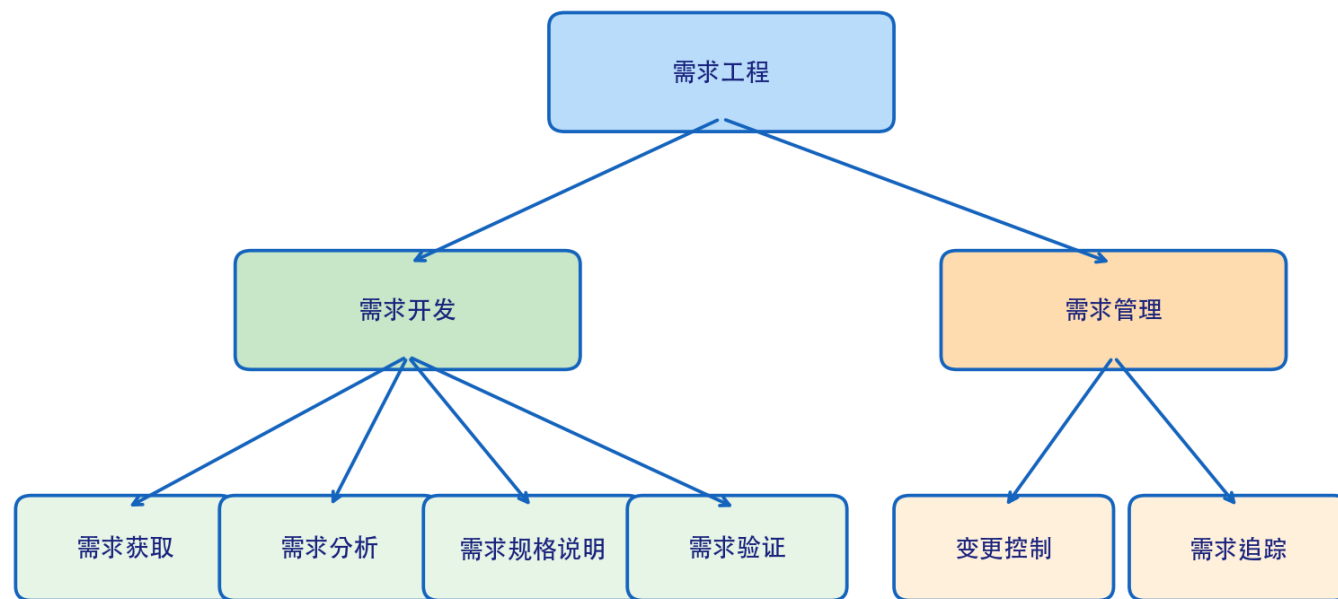
核心活动

- **收集信息**：从利益相关人和问题域中获取原始信息
- **分析问题**：理解问题的本质和边界
- **整合观点**：协调不同利益相关人的视角和诉求
- **记录需求**：形成规范的需求文档
- **验证正确性**：确保需求准确反映了利益相关人的真实期望

三个主要任务

1. 说明系统 **"做什么"** 和 **"为什么"**
2. 将需求映射为**可行的软件行为规格说明**
3. 处理需求的**变化**

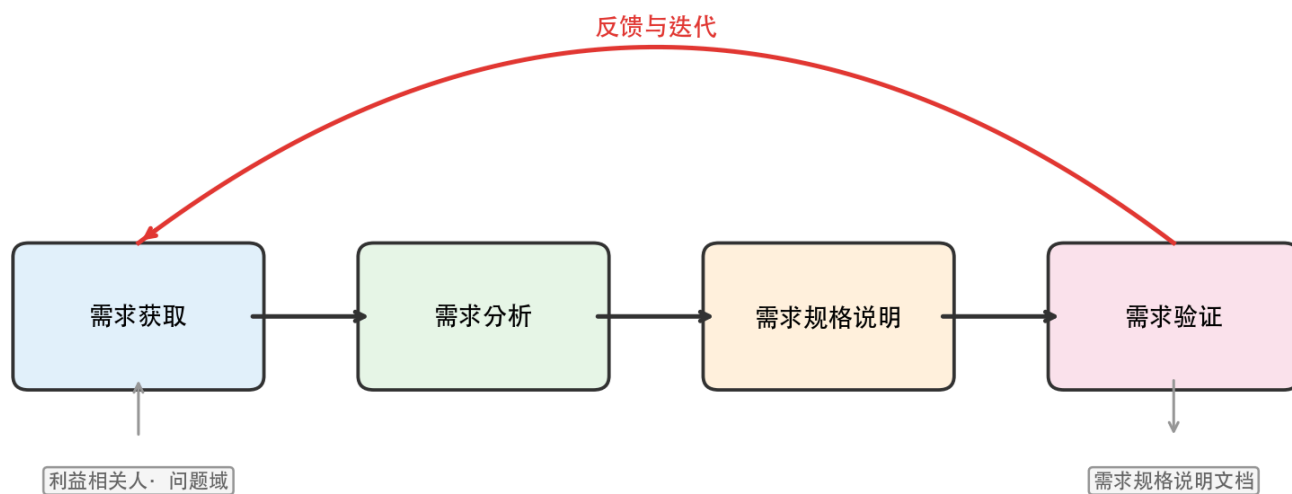
需求工程的活动 [SWEBOK 1.11]



需求工程的活动（续） [SWEBOK 1.11]

活动	目标
需求获取	从利益相关人和环境中收集原始需求
需求分析	对获取的需求进行建模、检测冲突与不一致
规格说明	将分析结果以标准化方式文档化
需求验证	确认需求的正确性、一致性、完整性
需求管理	管理需求变更、追踪需求状态、维护需求基线

需求开发过程模型 [SWEBOK 1.11]



💡 需求开发不是线性的——它是一个**不断迭代、逐步精化**的过程。在任何阶段发现问题，都可能需要回到前面的步骤重新处理。

PART 2 · 什么是需求？

需求的 IEEE 定义 [SWEBOK 1.1]

[IEEE 610.12-1990] 对"需求"给出了三层含义：

#	定义
1	用户为了解决问题或达到某些目标所需要的条件或能力
2	系统或系统部件为了满足合同、标准、规范或其它正式文档所规定的要求而需要具备的条件或能力
3	对 (1) 或 (2) 中的一个条件或一种能力的一种文档化表述

🔍 注意第三条——需求不仅是一种"期望"，更是一种可被记录、可被验证的表述。

需求的本质 [SWEBOK 1.1]

需求是一种期望

- 需求表达了人们对**问题得以解决后**所能达到的理想状态的憧憬

源自现实又高于现实

- 需求来源于现实世界中真实存在的**问题和痛点**
- 但需求描述的是**解决问题后的理想状态**，而非现状本身

需求是多变的、可调整的

- 随着对问题理解的深入，需求会**不断演化**
- 随着技术条件和商业环境的变化，需求可能被**重新定义**

💡 需求不是一成不变的"真理"，而是在特定语境下、对特定问题的**最佳期望表达**。

需求的表述 [SWEBOK 1.1]

作为一种期望，需求通常被表述为：

- "系统**应该**..."
- "在...时，系统**应该**..."
- "用户**可以通过**系统..."

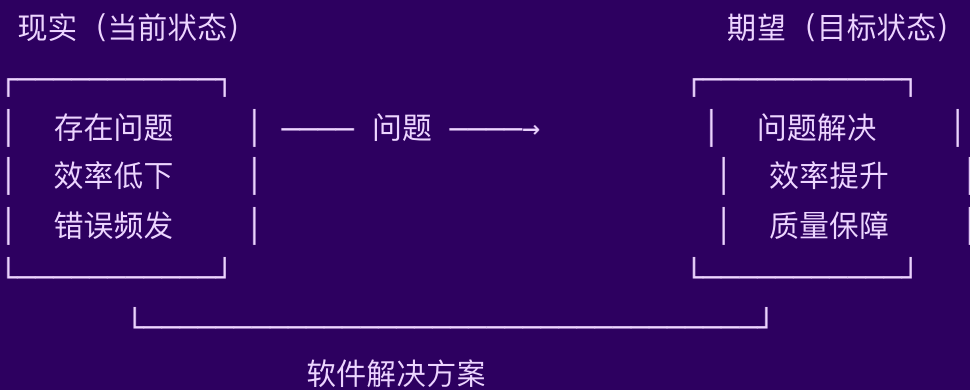
示例

R1: 系统应该允许顾客退回已经购买的产品。

要素	说明	R1 中的体现
主体	谁在使用系统	顾客
行为	系统应该做什么	允许退回产品
对象	操作针对什么	已购买的产品
约束	在什么条件下（可选）	（已购买）

软件解决方案 [SWEBOK 1.1]

- 需求是一种解决问题后所能达到的期望
- 问题是糟糕的一面，需求是理想的一面



💡 软件解决方案是连接问题现实与理想期望之间的桥梁。

问题域（Problem Domain） [SWEBOK 1.1]

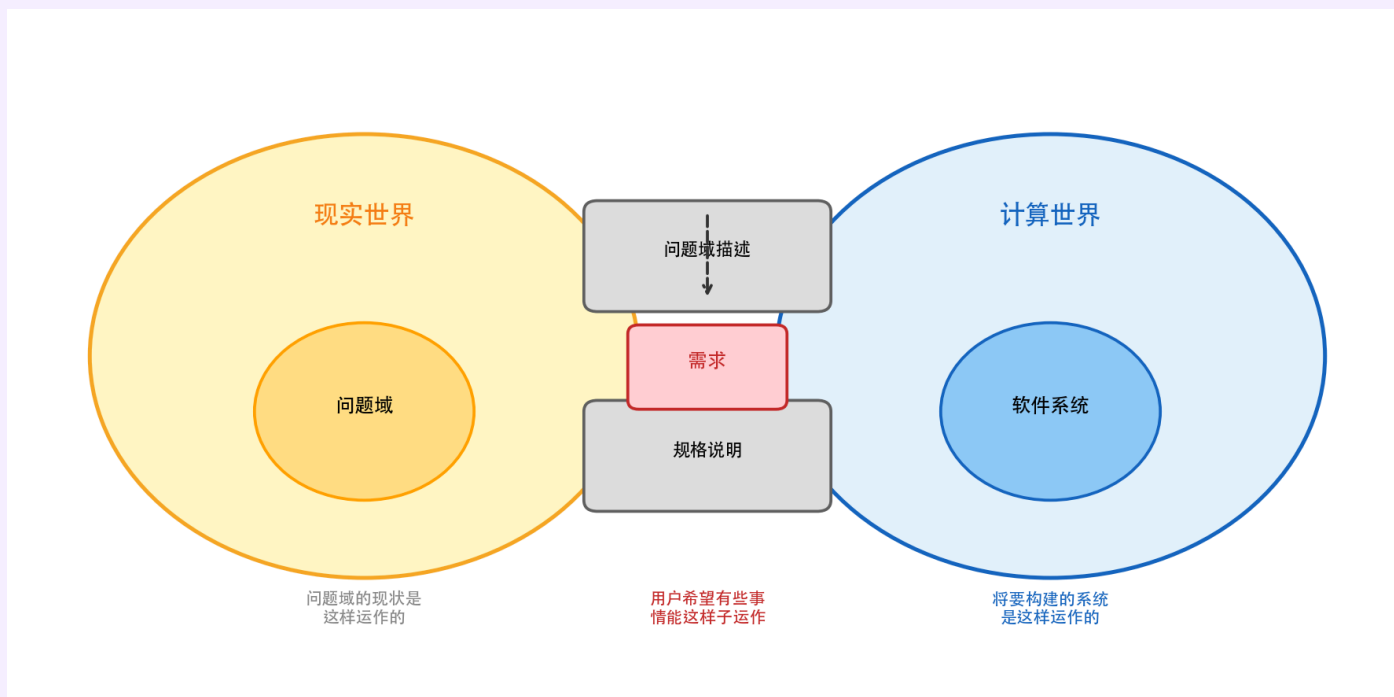
- **问题域**是现实世界运行规律的一种反映
- 需求的**产生地**，也是需求的**解决地**
- 软件开发必须**尊重问题域**——不能违背现实世界的规律和约束

问题域包含

要素	说明	示例
领域知识	该领域内通用的业务规则和事实	商品可退货期限为 7 天
利益相关人	与系统有利害关系的个人或组织	顾客、收银员、仓库管理员
现有流程	当前的工作方式和操作步骤	人工记账、纸质单据
约束条件	法律法规、行业标准、物理限制等	食品安全法、PCI 支付标准

🔍 脱离问题域谈需求，就像脱离地基建高楼——必然倒塌。

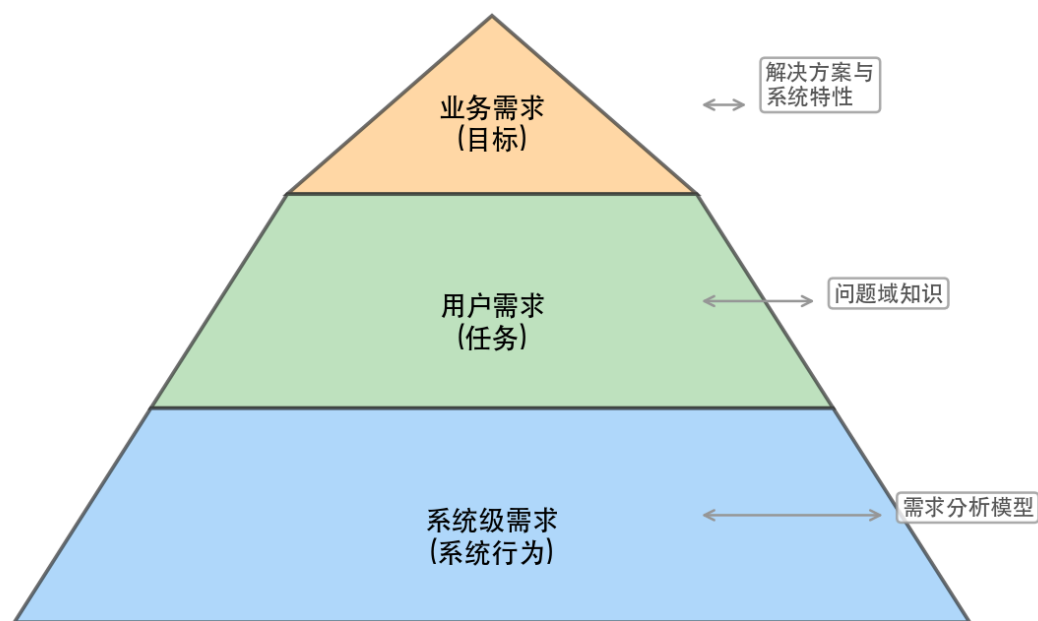
需求开发的目标 [SWEBOK 1.11]



需求开发的目标：将现实世界的**问题域**知识转化为计算世界可实现的**规格说明**。

PART 3 · 需求的层次与分类

需求的层次性 [SWEBOK 1.2, 1.9]



需求的层次性（续） [SWEBOK 1.2]

层次	对应产物	示例
业务需求	解决方案与系统特性	R2：使用 3 个月后，销售额提高 20%
用户需求	问题域知识	R3：系统要帮助收银员完成销售处理
系统级需求	需求分析模型	R4：收银员输入商品时，显示描述、单价、数量和总价

三个层次的需求示例 [SWEBOK 1.2]

业务需求 (Business Requirements)

R2: 在系统使用 3 个月后, 销售额度应该提高 20%。

用户需求 (User Requirements)

R3: 系统要帮助收银员完成销售处理。

系统级需求 (System Requirements)

R4: 收银员输入购买的商品时, 系统要显示该商品的描述、单价、数量和总价。

💡 从 R2 到 R4, 需求越来越**具体、可测试、可实现**。
但同时也越来越**远离业务目标**——三个层次缺一不可。

业务需求 (Business Requirements) [SWEBOK 1.2]

- 系统建立的**战略出发点**，表达为高层次的**目标 (Objective)**
- **系统特性 (Feature)** 定义了系统的**范围 (Scope)**
- 建立项目所有参与者的**共同前景 (Vision)**

要素	说明
目标	组织希望通过系统达成的商业成果
系统特性	系统应具备的高层次功能集合
范围	系统要做什么、不做什么的边界
前景文档	让所有利益相关人达成一致的共识文档

💬 业务需求回答的问题是："我们为什么要建这个系统？"

用户需求（User Requirements） [SWEBOK 1.2]

- 执行实际工作的用户对系统所能完成的**具体任务**的期望
- 描述用户通过系统可以**完成哪些活动**

用户需求的特性

特性	说明	示例
模糊不清晰	用户往往无法精确表达自己的需要	"我要一个好用的系统"
多特性混杂	一条需求中可能包含多个功能点	"系统要能查库存、下订单、打印报表"
多逻辑混杂	业务逻辑、异常处理、界面要求混在一起	"如果库存不足就提醒，并且界面要好看"

 用户需求是需求工程师需要深入挖掘和细化的对象。

系统级需求（System Requirements） [SWEBOK 1.9]

- 规格说明（Specification）：软件产品的方案描述
- 不是需求，但实现需求；不是问题域，但需要与问题域互动
- 以关注对外交互的方式描述软件解决方案

系统级需求的特点

特点	说明
精确性	消除歧义，使用结构化的语言描述
可测试性	每条需求都应该能被验证是否满足
可追溯性	能追溯到其对应的用户需求和业务需求
面向交互	描述系统与外部实体（用户、其他系统）的交互行为

💡 系统级需求是开发团队的行动指南，也是测试团队的验收依据。

从功能需求层次看需求开发 —— 连锁超市案例 [SWEBOK 1.2, 1.11]

阶段	内容
问题	连锁超市销售管理混乱，库存不准确，结账效率低
需求	提高销售额 20%，实现自动化库存管理，加速结账流程
问题域	超市业务流程、商品分类规则、促销策略、供应链关系
规格说明	收银员扫描商品 → 系统显示单价和总价 → 自动扣减库存 → 生成销售记录

问题（现实痛点）→ 需求（理想期望）→ 问题域（领域知识）→ 规格说明（系统方案）



需求的图谱（Spectrum） [SWEBOK 1.2, 1.3]

分类	说明	示例
项目需求	对项目成本、进度等约束	R5: 成本控制在60万以下； R6: 6个月内完成
过程需求	对开发过程的约束	R7: 需提交SRS、设计文档和测试报告； R8: 使用持续集成
系统需求	软件需求 + 硬件需求 + 其他	R9: 购买专用服务器； R10: 用户培训1周
不切实际的期望	超出技术能力的需求	R11: 预测会员未来购买商品； R13: 收银员2小时完成所有销售处理

💡 需求工程师的一个重要职责是**识别不切实际的期望**（如R11），将其转化为可实现的形式（如R14: 如果一个销售处理任务在2小时内没有完成，系统要撤销该任务的所有已执行操作）。

需求分类 [SWEBOK 1.4, 1.5]

功能需求 (Functional Requirements)

- 系统**应该做什么**
- 描述系统必须提供的功能和行为
- 示例：系统允许顾客退回已购买的产品

非功能需求 (Non-Functional Requirements)

- 系统**应该做到什么程度**
- 描述系统的质量属性和约束条件
- 示例：查询响应时间不超过 3 秒

💡 功能需求回答**"做不做"**，非功能需求回答**"做得好不好"**。

两者同等重要——一个不能退货的系统是失败的，一个退货要等 10 分钟的系统同样是失败的。

常见需求分类一览 [SWEBOK 1.4, 1.5]

类型	描述	示例
功能需求	系统必须提供的功能	系统允许顾客退回已购买的产品
性能需求	响应时间、吞吐量等	查询响应时间 < 3 秒
安全需求	访问控制、数据保护	用户密码必须加密存储
可用性需求	易学性、易用性	新用户 10 分钟内学会基本操作
可靠性需求	系统可用时间、故障恢复	系统可用性达到 99.9%
可维护性需求	修改和扩展的容易程度	新增支付方式不超过 5 人日
可移植性需求	跨平台、跨环境运行能力	系统需同时支持 Windows 和 Linux

🔍 非功能需求常常被忽视，但它们往往决定了系统的**用户满意度**和**长期可持续性**。

性能需求详解 [SWEBOK 1.7]

性能维度	定义	示例
速度 (Speed)	系统完成任务的时间	PR1: 所有用户查询必须在10秒内完成
容量 (Capacity)	系统能存储的数据量	PR2: 系统应能存储至少100万个销售信息
吞吐量 (Throughput)	连续时间内完成的事务数量	PR3: 解释器每分钟至少解析5000条无错误语句
负载 (Load)	系统可承载的并发工作量	PR4: 允许50个营业服务器同时上传下载
实时性 (Time-Critical)	严格的实时要求	PR5: 监测到病人异常后, 监控器必须在0.5秒内发出警报

需求的灵活性 [SWEBOK 1.7, 3.2]

- **PR6:** 98%的查询不能超过10秒。(量化但有弹性)
- **PR7** 三个层次：
 - (最低标准) 在200个用户并发时，系统不能崩溃
 - (一般标准) 在200个用户并发时，系统应该在80%的时间内能正常工作
 - (理想标准) 在200个用户并发时，系统应该能保持正常的工作状态

💡 设定需求的灵活性（最低/一般/理想），可以帮助团队在成本和质量之间做出权衡。

常见质量属性与案例 [SWEBOK 1.7]

质量属性	定义	超市系统案例
可靠性 (Reliability)	规格条件下执行所要求能力	QA1: 网络故障时, 系统不能出现故障
可用性 (Availability)	可操作和可访问的概率	QA2: 系统可用性达到98%
安全性 (Security)	阻止未授权访问	QA3: VIP顾客只能查看自己的信息
可维护性 (Maintainability)	修改以排障/改进的容易程度	QA4: 新增特价类型在2人月内完成
可移植性 (Portability)	跨环境转换能力	QA5: 1人月内从Windows换到Solaris
易用性 (Usability)	用户使用的努力和满意度	QA6: 使用1月的收银员效率达10件/分钟

💡 [Robert1990]: 在决定系统成功或失败的因素中, 满足非功能属性需求往往比满足功能性需求更为重要。

标准演进：ISO/IEC 9126 → 25010:2023 [SWEBOK 1.7]

标准	年代	质量特性数	关键变化
ISO/IEC 9126	1991	6	功能性、可靠性、易用性、效率、可维护性、可移植性
ISO/IEC 25010:2011	2011	8	新增 安全性（Security） 和 兼容性（Compatibility）
ISO/IEC 25010:2023	2023	8	细化子特性，增加**交互性（Interaction Capability）**替代易用性

2025 前沿：LLM + ISO 25010 自动推断非功能需求

- 通过细粒度提示框架，LLM 可从功能需求**自动推断**对应的非功能需求
- 按 ISO/IEC 25010:2023 分类，**80.4%** 的分类结果与人工一致
- 专家对推断质量的中位评分：**5.0/5**（median，>50% 专家给满分，但也有低分）

来源：Advancing RE with LLMs, ScienceDirect 2025
([sciencedirect.com/science/article/pii/S221282712500873X](https://www.sciencedirect.com/science/article/pii/S221282712500873X))

产品需求 vs 项目需求 [SWEBOK 1.3]

维度	产品需求 (Product)	项目需求 (Project)
关注点	软件应有的形态、功能和质量	构建软件的过程约束
别名	Software Product Requirements	Process / Business Requirements
示例	系统应支持 1000 并发用户	项目预算不超过 50 万
	查询响应 < 3 秒	6 个月内上线
	支持 iOS 和 Android	团队不超过 8 人
文档位置	SRS	项目章程 / 项目计划

💡 本课程重点讲授**产品需求**。但在实际项目中，项目需求（成本、进度、人员）同样约束着需求决策。

非功能需求深化 [SWEBOK 1.6–1.7]

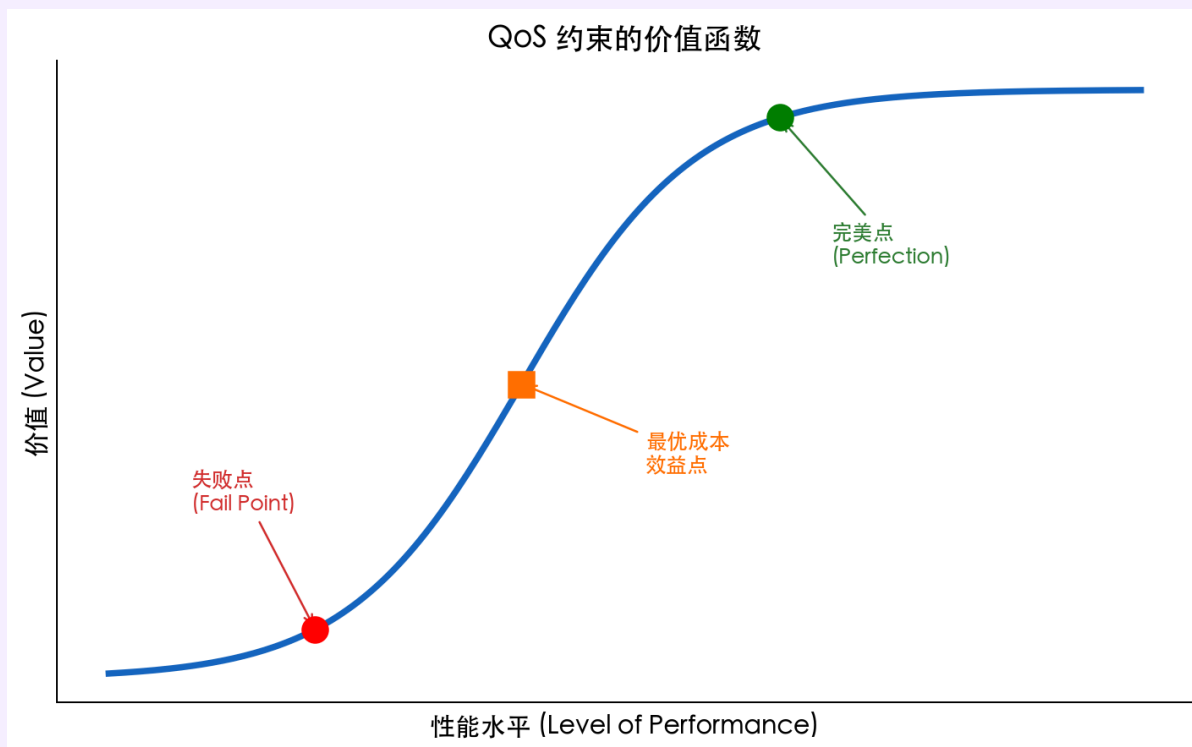
技术约束 (Technology Constraints)

- 强制或禁止使用特定技术
- 示例：必须使用 Java、必须运行在 Linux、禁止使用 GPL 库

服务质量约束 (Quality of Service Constraints)

- 不约束技术选择，而是约束可接受的性能水平
- 示例：响应时间、吞吐量、准确率、可靠性、可扩展性

QoS 约束的经济学 [SWEBOK 3.2]



💡 非功能需求的量化指标（如“响应 < 3 秒”）往往是**拍脑袋**定的。
理解 Perfection Point 和 Fail Point 能帮助做出更理性的决策。

派生需求 (Derived Requirements) [SWEBOK 1.10]

需求是上下文相关的

- 同一个决策，从不同角度可能是"需求"也可能是"设计决策"

示例：架构师决定采用管道-过滤器架构

视角	它是什么？
对整体项目（外部利益相关人视角）	一个 设计决策 ，不是需求
对子团队（负责开发过滤器的团队）	一个 派生需求 ，必须遵守

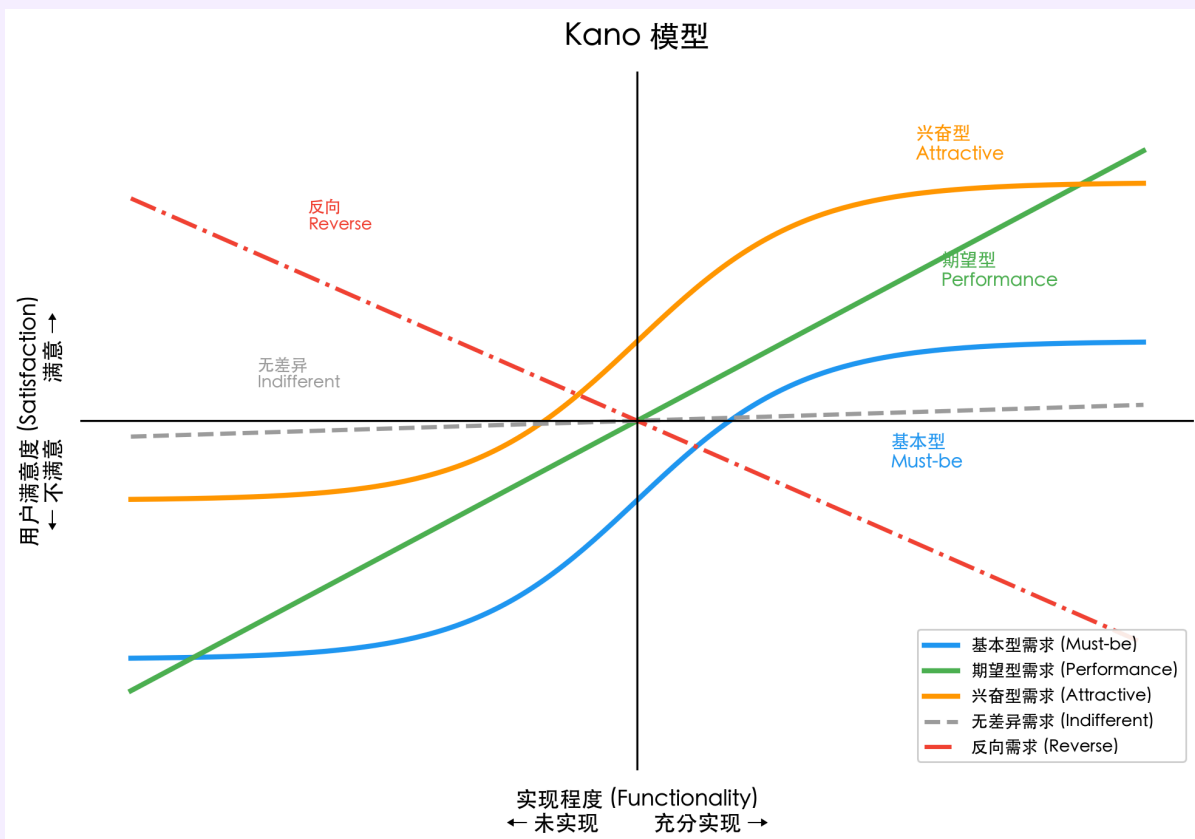
💡 航空航天行业用 **derived requirement** 表示"不是外部利益相关人提出的，而是内部决策产生的需求"。理解这一点有助于在大型项目中正确定位需求层次。

需求优先级排序 [SWEBOK 7.2]

为什么需要优先级?

- 资源有限，不可能同时实现所有需求
- 优先级帮助决定**先做什么、砍掉什么**

Kano 模型：理解用户满意度



需求优先级排序（续）[SWEBOK 7.2]

优先级计算公式（一种常见方法）

$$\text{Priority} = \text{Value} \times (1 - \text{Risk}) / \text{Cost}$$

- **Value**: 利益相关人感知价值
- **Risk**: 技术实现风险
- **Cost**: 实现成本

PART 4 · AI 时代的需求工程新挑战

LLM 在需求工程中的现状（2025 前沿数据）

指标	数据	来源
LLM4RE 研究增长	2024 年比 2023 年增长 136%	Frontiers 综述 2025
GPT-4 意图对齐度	比人工高 1.12 分	同上
GPT-4 完整度	比人工高 10.2%	同上
生成速度	人工的 720 倍	同上
生成成本	仅为人工的 0.06%	同上
预期需求捕获率	约 70%	同上

💡 AI 在需求生成的**速度和成本**上优势巨大，但在**完整性**上仍有明显缺口。

AI 能力边界 —— 核心矛盾（一）

AI 最大的短板不是"生成不了"，而是"你不知道它遗漏了什么"

- 30% 的需求遗漏 = 隐式约束 + 边界条件 + 领域知识
- 这些恰恰是最容易引发严重缺陷的需求类型
- 回忆 PART 1 的案例：737 MAX 的悲剧正是源于被遗漏的非功能约束

课程核心理念：

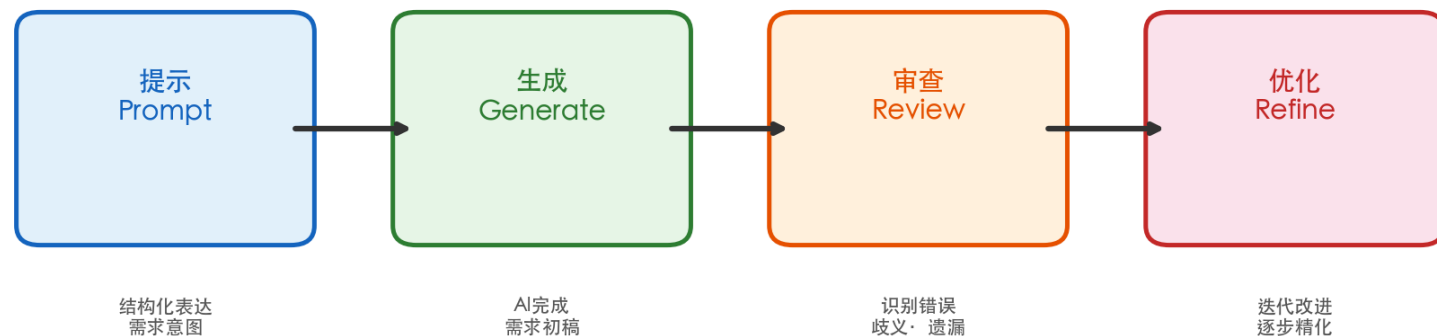
"学生最大的短板不是用不好 AI，而是不知道什么时候不能信 AI。"

AI 能力边界 —— 核心矛盾（二）

AI 擅长	AI 不擅长
快速生成结构化需求文档	发现隐式领域约束
从模板中泛化出新需求	识别利益相关人之间的矛盾
检测文本中的歧义和不一致	判断需求的商业优先级
生成标准化的需求表述	理解组织文化和政治因素

 **记住这张表**——后续 4 节课的每一个 AI 实践环节，你都需要判断当前任务落在哪一列。

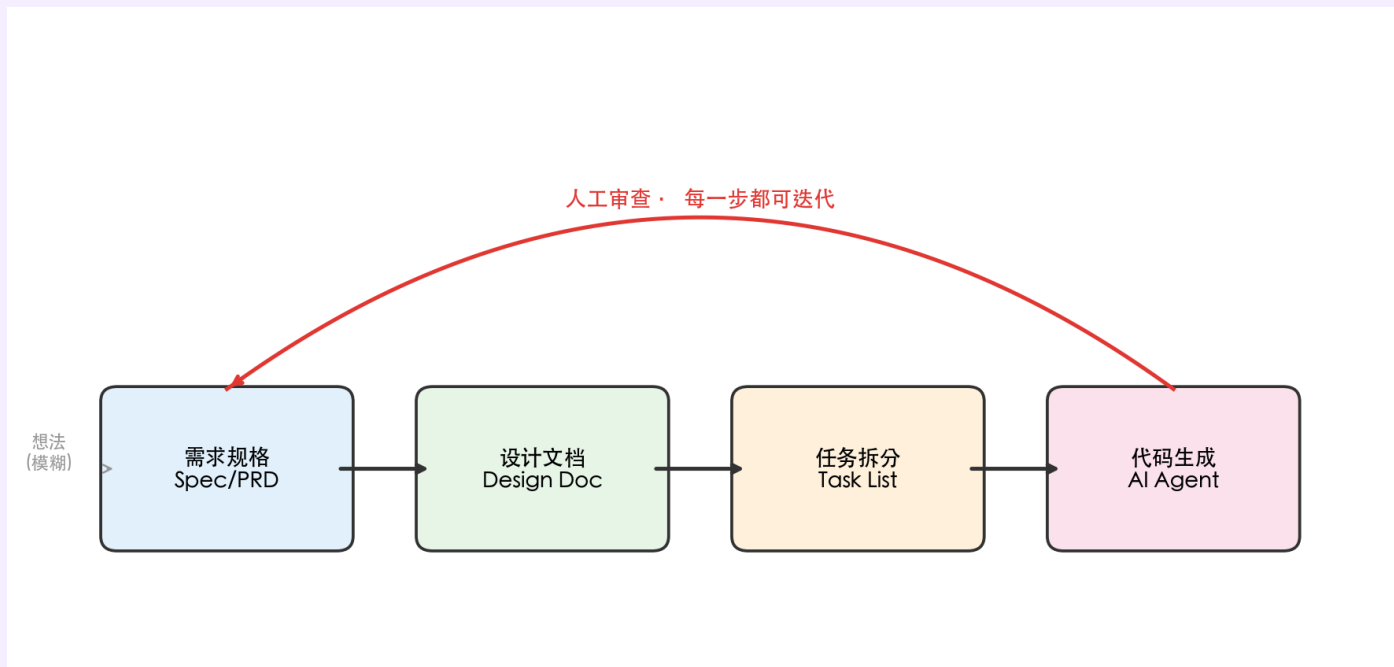
课程"提示 → 生成 → 审查 → 优化"四步范式



- **提示 (Prompt)**: 用结构化方式向 AI 表达需求意图
- **生成 (Generate)**: AI 快速生成需求初稿
- **审查 (Review)**: 人工识别 AI 产出中的错误、歧义和遗漏
- **优化 (Refine)**: 基于审查结果迭代改进需求

工业界趋势预告：Spec-Driven Development

2025 年，工业界出现了一个重要趋势——Spec 驱动开发 (SDD)：



- **GitHub Spec Kit** (2025)：开源 SDD 工具包，支持 22+ AI Agent 平台
- **AWS Kiro** (2025)：将 Spec 定义为 "超级 Prompt"
- **Claude Code** (Anthropic)：CLAUDE.md 作为项目"宪法"

Spec 案例：智能图书管理系统

模糊需求（传统方式）：

"做一个图书管理系统，能借还书就行。"

结构化 Spec（SDD 方式）：

字段	内容
Problem	图书馆人工登记借还效率低，错借率高达 5%，高峰期排队超 15 分钟
Solution	扫码借还 + 自动逾期提醒 + 实时库存查询
User Story	As a 学生, I want to 扫描学生证自助借书, So that 无需排队等候
Acceptance	Given 学生扫描学生证, When 借阅数未超上限, Then 3秒内完成借阅登记
Out of Scope	不实现推荐算法、不支持电子书借阅

💡 同一个需求，模糊版本让 AI 自由发挥，结构化版本约束 AI 的输出边界。

Spec 案例：CLAUDE.md —— 项目级 Spec

Anthropic 团队使用 `CLAUDE.md` 作为 AI Agent 的项目"宪法":

CLAUDE.md 章节	内容示例（在线考试系统）	对应需求概念
项目约束	Python 3.12 + FastAPI; PostgreSQL 16	技术约束 [SWEBOK 1.6]
编码规范	函数 ≤ 30行; 公开API须有OpenAPI文档	过程需求
绝对禁止 (Never Do)	禁止硬编码密码; 禁止跳过测试	安全约束 + 质量门禁
需要确认 (Ask First)	删表前确认; 改API签名前确认	变更控制 [SWEBOK 6.2]
测试要求	所有API须有 pytest 集成测试	验收标准 [SWEBOK 4.3]

💡 CLAUDE.md 本质上是**非功能需求 + 项目约束**的结构化表达——你在 L1 学到的概念，这里直接应用。

Spec 案例：Spec.md —— 功能级规格说明

与项目级的 CLAUDE.md 不同，Spec.md 描述单个功能的详细规格：

Spec: 用户注册与登录

背景

在线考试系统需要支持学生和教师两种角色的注册与登录。

功能需求

1. 学生使用学号 + 密码注册，教师使用工号 + 邀请码注册
2. 登录后根据角色跳转到不同的首页
3. 连续 5 次登录失败后锁定账户 30 分钟

接口定义

- POST /api/auth/register { role, id, password }
- POST /api/auth/login { id, password }
- GET /api/auth/me → { role, name, ... }

验收标准

- Given 学生输入正确学号和密码, When 点击登录, Then 跳转到学生首页
- Given 用户连续输入错误密码5次, When 第6次尝试, Then 返回"账户已锁定"

预习测试 / 课堂练习

1. 需求的分类有哪些？

2. 需求的层次性是什么？

3. 以下需求表述是否正确？如果不正确请改正：

"在使用选课系统时，学生必须要在**一刻钟**内完成一个选课处理的所有操作。"

"所有的用户查询都必须在**很快的时间**内完成。"

 提示：好的需求应该是**明确的、可测试的、无歧义的**。

"一刻钟"是否明确？"很快"是否可测试？

本节课结束

"需求是软件与现实世界的契约。"